ACIBADEM UNIVERSITY

CSE 322 – CLOUD COMPUTING

ASSIGNMENT

ACU AI Chatbot

A Django Web Application with Local LLM Integration
Containerized with Docker & Docker Compose

Spring 2026

1. Assignment Overview

In this group assignment you will build an **AI-powered chatbot web application** that can answer questions about **Acıbadem University** using data collected from the university's websites. The chatbot will be powered by an **open-source Large Language Model (LLM)** running locally on your machine (e.g., Google Gemma, Meta LLaMA, Microsoft Phi, Mistral, etc.).

The application will be built with **Django** and **PostgreSQL**, and the entire system — including the AI model — must be packaged and run using **Docker and Docker Compose**. The project should be startable with a single **`docker-compose up`** command.

The AI component must work correctly. Your chatbot should be able to answer questions about Acıbadem University accurately and relevantly based on the data you've collected from the university's websites. This requires selecting an appropriate open-source model and customizing it at least at the prompt engineering level so that it produces accurate, on-topic responses. The main course topic is cloud computing and containerization, but the AI must function properly — not just be connected.

2. System Architecture

Your system must consist of the following components, each running as a **separate Docker container** and orchestrated via a single `docker-compose.yml` file:

Component	Technology	Description
Web Application	Django 4.x / 5.x	Chat interface where users ask questions. REST API backend that handles queries, retrieves context, and communicates with the LLM.
Database	PostgreSQL 15+	Stores collected Acıbadem University website content, chat history, and application data.
LLM Service	Any open-source LLM	A locally running AI model served via an HTTP API. Django sends prompts with context and receives generated answers.

Note: You may add additional containers if your design requires them (e.g., Nginx as a reverse proxy, Redis for caching, a vector database for semantic search). These are not mandatory but may earn bonus points.

2.1 How It Works

Your chatbot can fetch data either **ahead of time** (batch scraping and storing in the database) or **on the fly** (fetching relevant pages in real time when a question is asked), or a combination of both. The important thing is that **the chatbot answers questions using real data from Acıbadem University's websites**.

A typical flow looks like this:

1. The user types a question in the chat interface.
2. Django retrieves relevant data (from the database, or by fetching it live from the Acıbadem websites, or both).
3. The retrieved context + user question are sent to the local LLM as a prompt.
4. The LLM generates a natural-language answer, which is displayed to the user.

3. Requirements

3.1 Django Web Application

Build a Django application with the following features:

- A chat interface where users can type questions and receive AI-generated answers. It can be as simple as a single-page form with a text input and a response area.
- Chat history: store past conversations in PostgreSQL so users can see their previous questions and answers.
- An admin panel (Django Admin is sufficient) to view collected data, chat logs, and basic system status.
- At least one REST API endpoint (e.g., POST /api/chat/) that accepts a question and returns an answer. You may use Django REST Framework or Django's built-in JsonResponse.
- Basic error handling: if the LLM service is unavailable or slow, show a user-friendly message instead of crashing.

3.2 Website Data Collection

Your chatbot needs a knowledge base to answer questions. You should collect content from Acıbadem University's websites and store it in your PostgreSQL database, or retrieve it in real time when needed.

Data Sources

There are two primary data sources you can use. Both are publicly accessible and do not require login:

- **https://www.acibadem.edu.tr –** The main university website. Contains general information about the university, faculties, departments, campus life, admission, announcements, contact details, and more.
- **https://obs.acibadem.edu.tr –** The Bologna Information System / student portal (public section). Contains detailed academic program information: course catalogs, curricula, course descriptions, credit hours, learning outcomes, etc. For example, the Computer Engineering program page is available at: https://obs.acibadem.edu.tr/oibs/bologna/index.aspx?lang=tr&curOp=showPac&curUnit=14&curSunit=6246#

Important: Only collect **publicly available data** from these sources. Do ***not*** attempt to log in or access any restricted/authenticated areas of these systems.

There are no strict rules on how you collect this data. You are free to choose your approach:

- Write a Python script using requests + BeautifulSoup.
- Use a framework like Scrapy.
- Manually curate content into seed data files or fixtures.

- Use Selenium or Playwright for JavaScript-heavy pages (the Bologna system may require this).
- Fetch data on the fly when a user asks a question.
- Any combination of the above.

The important thing is that your chatbot has access to **real Acıbadem University content** (e.g., program descriptions, course info, admission details, contact information, campus facilities, FAQ, etc.) and uses it as context when generating answers.

> ⚠️ **Important – Responsible Scraping**
>
> When collecting data from the university websites, you must act responsibly:
>
> - Respect each website's terms of service.
> - Do NOT send excessive or rapid-fire requests. Add appropriate delays between requests (e.g., 1–2 seconds).
> - Do NOT implement continuous/aggressive refresh loops that could be perceived as a DoS attack.
>
> **Collect data responsibly and in compliance with the website's rules. Excessive or abusive scraping may get your IP blocked by the university's IT department.**

3.3 Local LLM Integration

Your chatbot must use an open-source LLM that runs **entirely locally inside a Docker container**. No external API calls to OpenAI, Google, Anthropic, or any paid AI service are allowed.

The AI must produce accurate, relevant answers. It is not sufficient to simply connect to a model and pass raw questions to it. You need to ensure that when a user asks about Acıbadem University, the chatbot provides correct, on-topic responses grounded in real website data.

Open-Source Model Examples

Any open-source, self-deployable model is acceptable. Below are some well-known options:

Model	Parameters	Approx. Size	Notes
Google Gemma	2B / 7B	~1.5–5 GB	Lightweight and efficient
Meta LLaMA 3.x	3B / 8B	~2–5 GB	Strong general-purpose model
Microsoft Phi-3	3.8B / 14B	~2–8 GB	Compact but capable
Mistral	7B	~4 GB	High quality for its size
Qwen 2.5	3B / 7B	~2–5 GB	Multilingual support
DeepSeek R1 (distilled)	1.5B / 7B	~1–5 GB	Reasoning-focused

The specific model you choose does not matter as much as **the results it produces**. Pick the model that works best for your use case and hardware. A small model that gives accurate answers is better than a large model that gives irrelevant ones.

How to Serve a Model Locally

There are several tools for serving open-source models inside Docker:

- Ollama (ollama.ai) – Simple Docker image with HTTP API. Pull and run models with one command.
- vLLM – High-performance inference server, ideal for larger models.
- llama.cpp – Lightweight C++ inference engine, works well on CPU-only machines.
- LocalAI – Drop-in OpenAI-compatible API for local models.
- Hugging Face TGI (Text Generation Inference) – Production-grade model serving.

A typical integration works like this:

1. Add the LLM service to your `docker-compose.yml` as a container.
2. The model is either pulled at container startup or baked into a custom image.
3. Your Django app sends HTTP requests to the LLM container's API with a prompt (context + question).
4. The LLM returns a generated answer, which Django passes back to the user.

3.4 AI Customization (Required)

Simply connecting to a model is not enough. You are expected to **customize the AI at least at the prompt engineering level** to ensure it gives accurate and relevant answers about the university. This is a required part of the assignment, not optional.

What is expected:

- **Prompt Engineering:** Design system prompts and user prompts that include the relevant Acıbadem University context and instruct the model to answer based on that context.
- **Testing & Iteration:** Test your prompts with various types of questions (academic programs, courses, admission, campus life, contact info, etc.) and iterate until the model produces satisfactory responses.
- **Adjustment:** If the model struggles with certain types of questions, adjust your prompting strategy, try a different model, or fine-tune using tools like Ollama's custom Modelfile, LoRA, or QLoRA. The level of customization is up to you, but the end result must be a chatbot that works.

Remember: this is a cloud computing course, not an AI course. We are not asking you to train a model from scratch. But the AI component of your project must **function correctly and produce useful answers**. A chatbot that just returns generic or irrelevant responses will not receive full credit.

4. Deliverables

4.1 Source Code Repository

- A GitHub repository shared with the instructor/TA.
- Your team name and repository URL must be entered in the shared course spreadsheet.
- A clear and helpful README.md with setup instructions, architecture overview, and team member info.
- Meaningful Git history showing contributions from all team members.

Suggested Project Structure

```
acibadem-chatbot/
├── docker-compose.yml
├── .env.example
├── README.md
├── webapp/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── manage.py
│   ├── config/           # Django settings, urls, wsgi
│   ├── chat/             # Chat app (views, models, templates)
│   ├── scraper/          # Data loading scripts or management commands
│   ├── static/
│   └── templates/
└── docs/
    └── report.pdf
```

4.2 Technical Report

A written report (6–8 pages, PDF) covering:

1. **Introduction:** Problem statement and project objectives.
2. **Architecture:** Architecture diagram showing containers, data flow, and inter-service communication.
3. **Implementation:** Technology choices and justification (framework, database, LLM model, serving tool).
4. **Docker Setup:** Dockerfile and Docker Compose walkthrough — explain your containerization decisions.
5. **AI Integration:** Which model you chose, how you customized/tuned it, prompting strategy, and a quality assessment of the output.
6. **Evaluation:** At least 10 sample questions and the chatbot's answers. Discuss accuracy.
7. **Challenges:** What went wrong, how you resolved it, what you'd do differently.
8. **Team Contributions:** Clear breakdown of who did what, backed by Git commit history.

The report is submitted twice: first as a **proposal** (initial version covering your planned approach) and then as a **final report** (complete version with evaluation results). See Section 5 for the timeline.

4.3 Live Demonstration

- 10 - 15 minute live demo followed by Q&A.
- Show the system running via Docker Compose.
- Demonstrate: starting containers, asking questions, the LLM generating accurate answers, chat history, admin panel.
- All group members must present and be prepared to answer questions about any part of the code.

5. Project Timeline

Milestone	When	What You Deliver
Proposal	Week 8-9	Project proposal presentation: planned architecture, technology choices, team role distribution, initial progress (Docker Compose skeleton, model selection). Submit the initial version of your report.
Final	Week 14-15	Complete, working application. Live demonstration of all features: Docker containers running, chatbot answering questions accurately, chat history, admin panel. Submit the final report and GitHub repository.

6. Bonus Points

The following optional enhancements can earn extra credit. These should only be attempted after the core requirements are fully working.

Bonus Item	Description
Cloud Deployment	Deploy your application to any cloud provider (GCP, AWS, Azure, etc.). A deployment-ready project with clear documentation can also earn credit even without a live deployment.
Kubernetes	Kubernetes manifests (Deployments, Services, PVCs); deployed on Minikube or a cloud K8s cluster.
RAG with Vector Database	Semantic search using a vector database (ChromaDB, Qdrant, pgvector) with embeddings instead of keyword matching.
Advanced AI Fine-Tuning	Deep fine-tuning beyond basic prompt engineering (LoRA, QLoRA, custom training data). Experiments and results documented.
CI/CD Pipeline	GitHub Actions or similar for automated testing and Docker image builds.
Additional Enhancements	Nginx reverse proxy, streaming responses (SSE/WebSocket), monitoring (Prometheus/Grafana), user authentication, etc.

7. Group Policy & Academic Integrity

- **Group Size:** 3–4 students per group.
- **Contribution Tracking:** Every member must have meaningful Git commits. The report must include an individual contribution section.
- **Peer Evaluation:** Each member will submit a confidential peer evaluation form. Significant contribution imbalances may result in individual grade adjustments.
- **Academic Integrity:** You may use AI assistants (Copilot, ChatGPT, etc.) as learning aids, but you must understand and be able to explain every line of your code during the demo. Copying code from other groups or submitting work you cannot explain will result in zero credit.

8. Useful Resources

Core:

- Docker Compose Documentation: https://docs.docker.com/compose
- Django Documentation: https://docs.djangoproject.com
- Django REST Framework: https://www.django-rest-framework.org
- PostgreSQL Full-Text Search: https://www.postgresql.org/docs/current/textsearch.html

LLM Serving:

- Ollama: https://ollama.ai/docs
- vLLM: https://docs.vllm.ai
- llama.cpp: https://github.com/ggerganov/llama.cpp
- Hugging Face Model Hub: https://huggingface.co/models

Fine-Tuning & RAG:

- LoRA Fine-Tuning Guide: https://huggingface.co/docs/peft
- Ollama Modelfile (Custom Models): https://github.com/ollama/ollama/blob/main/docs/modelfile.md
- ChromaDB: https://docs.trychroma.com
- LangChain: https://python.langchain.com

Good luck!